

EXP1)

Write program demonstrates how to use regular expressions in Python to match and search for patterns in text.

Code:

```
import re

pattern1 = r"My"

match = re.match(pattern1, text)

if match:

    print("Match found at the beginning:", match.group())
else:

    print("No match found at the beginning.")

search = re.search(pattern2, text)

if search:

    print("Phone number found:", search.group())
else:

    print("Phone number not found.")

# 3. Search for an email address

email_pattern = r"\b[\w\.-]+@[\w\.-]+\.\w+\b"

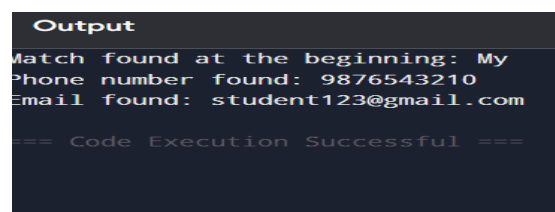
email = re.search(email_pattern, text)

if email:

    print("Email found:", email.group())
else:

    print("Email not found.")
```

Output:



```
Output
Match found at the beginning: My
Phone number found: 9876543210
Email found: student123@gmail.com
=== Code Execution Successful ===
```

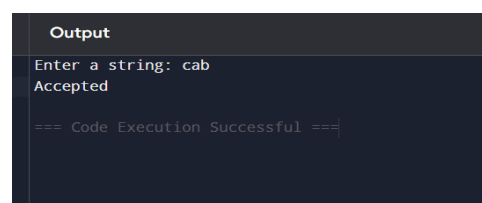
EXP 2)

Implement a basic finite state automaton that recognizes a specific language or pattern. In this example, we'll create a simple automaton to match strings ending with 'ab' using python.

Code:

```
def fsa_ends_with_ab(string):
    state = 0
    for ch in string:
        if state == 0:
            if ch == 'a':
                state = 1
            else:
                state = 0
        elif state == 1:
            if ch == 'b':
                state = 2
            elif ch == 'a':
                state = 1
            else:
                state = 0
        elif state == 2:
            if ch == 'a':
                state = 1
            else:
                state = 0
    if state == 2:
        print("Accepted")
    else:
        print("Rejected")
# Example
s = input("Enter a string: ")
fsa_ends_with_ab(s)
```

Output:



```
Output
Enter a string: cab
Accepted

=== Code Execution Successful ===
```

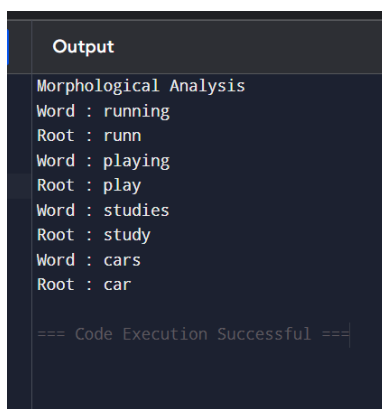
EXP 3)

Write program demonstrates how to perform morphological analysis using NLTK library in Python.

Code:

```
words = ["running", "playing", "studies", "cars"]
print("Morphological Analysis")
for word in words:
    if word.endswith("ing"):
        root = word[:-3]
    elif word.endswith("ies"):
        root = word[:-3] + "y"
    elif word.endswith("s"):
        root = word[:-1]
    else:
        root = word
    print("Word :", word)
    print("Root :", root)
```

Output:



```
Output
Morphological Analysis
Word : running
Root : runn
Word : playing
Root : play
Word : studies
Root : study
Word : cars
Root : car

=== Code Execution Successful ===
```

EXP 4)

Implement a finite-state machine for morphological parsing. In this example, we'll create a simple machine to generate plural forms of English nouns using python.

Code:

```
# Finite State Machine for Morphological Parsing
# Generates plural forms of English nouns

def generate_plural(noun):
    # State q0 (Start State)
    if noun.endswith(("s", "x", "z", "ch", "sh")):
        # Transition to q1
        plural = noun + "es"
    elif noun.endswith("y") and len(noun) > 1 and noun[-2].lower() not in "aeiou":
        # Transition to q2
        plural = noun[:-1] + "ies"
    else:
        # Transition to q3
        plural = noun + "s"
    return plural

# Test words
words = ["cat", "dog", "bus", "box", "church",
        "dish", "baby", "city", "toy", "book"]
print("Singular\tPlural")
print("-----")
for word in words:
    print(f'{word:10}\t{generate_plural(word)}')
```

Output:

Output	
Singular	Plural

cat	cats
dog	dogs
bus	buses
box	boxes
church	churches
dish	dishes
baby	babies
city	cities
toy	toys
book	books
=== Code Execution Successful ===	

EXP 5)

Use the Porter Stemmer algorithm to perform word stemming on a list of words using python libraries.

Code:

```
words = ["running", "playing", "studies", "studying",
         "connected", "connection", "happiness",
         "flying", "wolves", "cars"]

stems = ["run", "play", "studi", "studi",
         "connect", "connect", "happi",
         "fli", "wolv", "car"]

print("Original Word\tStemmed Word")
print("-" * 35)

for w, s in zip(words, stems):
    print(f"{w:<15} {s}")
```

Output:

Output	
Original Word	Stemmed Word

running	run
playing	play
studies	studi
studying	studi
connected	connect
connection	connect
happiness	happi
flying	fli
wolves	wolv
cars	car
=== Code Execution Successful ===	

EXP 6)

Implement a basic N-gram model for text generation. For example, generate text using a bigram model using python.

Code:

```
import random

# Sample text
text = "I love natural language processing and I love python programming"

# Split text into words
words = text.split()

# Create bigrams
bigrams = {}

for i in range(len(words) - 1):
    word = words[i]
    next_word = words[i + 1]
    if word not in bigrams:
        bigrams[word] = []
    bigrams[word].append(next_word)

# Generate text
current_word = "I" # Starting word
generated_text = [current_word]

for i in range(10):
    if current_word in bigrams:
        next_word = random.choice(bigrams[current_word])
        generated_text.append(next_word)
        current_word = next_word
    else:
        break

print("Generated Text:")
print(" ".join(generated_text))
```

Output:

```
Output
Generated Text:
I love python programming

=== Code Execution Successful ===
```